

SYNERGY NETWORK

Synergy Protocol Naming & Encoding Standards

Consolidated Technical Standard

Standard Numbers: SNTS-01 / SNTS-04 / SNTS-06 / SNTS-07 / SNTS-08 / SNTS-09

Version: 1.0

Date: March 2026

Classification: Public Technical Standard

Status: Active

This document consolidates the canonical naming, formatting, and encoding conventions for all Synergy Network protocol entities.

Table of Contents

Table of Contents.....	2
Preamble.....	4
0.1 Purpose and Scope.....	4
0.2 How to Use This Document.....	4
0.3 Constituent Standards.....	4
0.4 Companion Standards.....	4
0.5 Revision History	4
Part I — Address Formatting (SNTS-01)	5
1.1 Overview	5
1.2 Address Type Registry	5
1.3 Non-Prefix Identifiers.....	7
1.4 Encoding and Key Generation	7
Part II — Denomination & Precision (SNTS-04)	8
2.1 Purpose and Scope.....	8
2.2 Monetary Parameters.....	8
2.3 Denomination Hierarchy.....	8
2.4 Accounting Rules	8
2.4.1 Ledger Storage	8
2.4.2 Smart Contract Arithmetic (AIVM)	9
2.4.3 Wallet & Explorer Display	9
2.5 Gas Model	9
2.6 Cross-Chain Precision Mapping.....	9
2.7 SDK Constants.....	9
2.7.1 Rust	9
2.7.2 JavaScript.....	10
2.8 Compliance Requirements	10
Part III — Post-Quantum Cryptography Naming Standards (SNTS-06)	11
3.1 Overview and Scope	11
3.2 General Rule: Documentation Form vs. Code Form	11
3.3 ML-KEM (Key Encapsulation — formerly CRYSTALS-Kyber)	11
3.4 ML-DSA (Digital Signatures — formerly CRYSTALS-Dilithium).....	12
3.5 FN-DSA (Digital Signatures — formerly Falcon).....	12
3.6 SLH-DSA (Hash-Based Signatures — formerly SPHINCS+).....	12

3.7 HQC-KEM (Code-Based KEM — formerly Hamming Quasi-Cyclic)	13
3.8 CMCE-KEM (Classic-McEliece — Code-Based KEM)	14
Part IV — Binary & Hexadecimal Encoding Conventions (SNTS-07)	15
4.1 Scope	15
4.2 General Hex Encoding Rules	15
4.3 Typed Hash Identifiers	15
4.4 Public Key Serialization	16
4.5 Signature Serialization	16
4.6 Encoding Formats Explicitly Not Used	16
Part V — Timestamp Representation (SNTS-08)	17
5.1 Wire Format	17
5.2 Precision Rationale	17
5.3 Display Format in Documentation and UI	17
5.4 Timestamp Field Naming Conventions	17
Part VI — Network Environment Identifier Conventions (SNTS-09)	18
6.1 Canonical Environment Names	18
6.2 Usage Rules by Context	18
6.3 Port Conventions	18
6.4 Global Configuration	19
Appendix A — Quick Reference	20
A.1 Address Prefixes (Selected)	20
A.2 Denomination Units	20
A.3 PQC Algorithm Identifiers (Active Protocol)	20
A.4 Hex Encoding Rules at a Glance	20
A.5 Timestamp Rules at a Glance	20
A.6 Network Environments at a Glance	21

Preamble

0.1 Purpose and Scope

This document is the authoritative reference for naming, formatting, and encoding conventions used across the Synergy Network protocol. It defines how protocol entities — addresses, monetary amounts, cryptographic algorithm identifiers, raw byte data, timestamps, and environment names — must be represented in all contexts: source code, documentation, APIs, configuration files, user interfaces, and inter-system communication.

Consistent representation of protocol entities is a prerequisite for interoperability, auditability, and correctness. Implementations that deviate from these conventions introduce ambiguity, silent compatibility failures, and documentation drift. All Synergy Network software, SDKs, wallets, explorers, and documentation must conform to the standards defined herein.

0.2 How to Use This Document

Each Part of this document is self-contained and addresses a specific category of protocol entity. Cross-references between Parts are made explicit where relevant. When a Part defines both a "documentation form" and a "code form" for the same entity, both are normative — the correct form depends on the context of use as defined in that Part.

Appendix A provides a single-page quick-reference cheat sheet summarising the most commonly needed conventions across all Parts.

0.3 Constituent Standards

This consolidated document incorporates and supersedes the following individual standards:

SNTS-01 Synergy Network Address Formatting Specification (Revised 12/06/2025)

SNTS-04 SNRG Denomination & Precision Specification v1.2 (December 2025)

SNTS-06 Post-Quantum Cryptography Naming & Formatting Standards

SNTS-07 Binary & Hexadecimal Encoding Conventions

SNTS-08 Timestamp Representation Standard

SNTS-09 Network Environment Identifier Conventions

0.4 Companion Standards

The following standards are referenced by this document but are maintained separately:

SNTS-03 AIVM Execution Determinism Standard

SNTS-05 Synergy Cross-Chain Protocol (SXCP) Specification

0.5 Revision History

Version	Date	Summary
1.0	March 2026	Initial consolidated release combining SNTS-01, SNTS-04, SNTS-06, SNTS-07, SNTS-08, and SNTS-09.

Part I — Address Formatting (SNTS-01)

1.1 Overview

Every protocol entity on the Synergy Network that requires a persistent, verifiable on-chain identity is assigned a typed address. All addresses use Bech32m encoding and are derived from FN-DSA-1024 (formerly Falcon-1024) quantum-safe keypairs. The address type is encoded in a human-readable prefix that precedes the Bech32m data, making the address type immediately legible to both humans and software without requiring additional metadata.

Bech32m encoding provides case-insensitive, error-detecting representations that eliminate the class of errors introduced by mixed-case checksum schemes. The encoding uses only alphanumeric characters from a restricted alphabet, making addresses safe in all contexts including URLs, QR codes, and printed material.

Unauthorized introduction of new address prefixes or repurposing of reserved prefixes, requires formal DAO approval. All changes to the prefix registry must be accompanied by an SNTS amendment.

1.2 Address Type Registry

The table below is the complete, authoritative prefix registry. Prefixes are grouped by entity category. The example address column uses placeholder characters to illustrate length and structure.

Prefix	Example Address	Use Case / Description
Wallet Addresses		
synw	synwl1fgerdqg1c6p74p9u6k8ghfss159q8jzhuwm07	Primary wallet prefix for standard user accounts. Integrates with UMA mapping.
syms	syms1xx	Secondary wallet prefix for utility or exchange wallets. Common for custodial or automated accounts.
syna	synalxxx	Standard wallet identifier. Formerly transaction prefix; reassigned as general account prefix.
synz	synzlxxx	Alternate wallet prefix for UMA-linked smart wallets. Used for imported or specialized multi-chain identities.
Transaction Identifiers		
syntxn	syntxn1xx	Standard on-chain transaction on the PoSy core layer. Used for intra-chain transfers, staking, and governance votes.
synxxn	synxxn1xx	SXCP cross-chain transaction identifier. Executed via the Synergy Cross-Chain Protocol.
synixn	synixn1xx	Reserved for internal infrastructure events. Future use for validator-relayer sync or system state updates.
Fungible Tokens (STS-9 Standard)		
synb1	synb11xx	STS-9 Fungible Token Standard — Tier 1.
synb2	synb21xx	STS-9 Fungible Token Standard — Tier 2.
synb3	synb31xx	STS-9 Fungible Token Standard — Tier 3.

Non-Fungible Tokens (STS-NF Standard)		
synn1	synn1xx	STS-NF Non-Fungible Token Standard — Type 1.
synn2	synn2xx	STS-NF Non-Fungible Token Standard — Type 2.
Multi-Asset Tokens (STS-MA Standard)		
synj	synj1xx	STS-MA Multi-Asset Token Standard.
Identity & Credential Tokens (STS-ID Standard)		
synk	synk1xx	STS-ID Identity & Credential Token Standard.
Smart Contracts		
synq	synq-contract-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx	System-level contract (governance, treasury, protocol). Deployed by DAO or core developers; audited and governance-controlled.
sync	sync-contract-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx	Custom contract deployed by developers or organizations. Application-layer logic on AIVM; no special privileges.
Node Addresses		
synv1	synv1xx	Class I — Consensus Nodes. Secure the chain through PoSy consensus.
synv2	synv2xx	Class II — Interoperability Nodes. Execute SXCP and UMA operations.
synv3	synv3xx	Class III — Intelligence & Computation Nodes. Provide decentralized compute and AI services.
synv4	synv4xx	Class IV — Governance & Treasury Nodes. Manage DAO proposals and treasury execution.
synv5	synv5xx	Class V — Service & Support Nodes. Operate ancillary infrastructure.
Node Cluster Addresses		
synggrp1	synggrp1xxx	Node cluster group 1.
synggrp2	synggrp2xxx	Node cluster group 2.
synggrp3	synggrp3xxx	Node cluster group 3.
synggrp4	synggrp4xxx	Node cluster group 4.
synggrp5	synggrp5xxx	Node cluster group 5.
DAO & Governance Addresses		
syndao	syndao1xxx	DAO governance proposal identifier. Created automatically for each submitted proposal.
syno	syno1xxx	Governance Oversight Auditor address. Belongs to Security Council or auditing committees.
syny	syny1xxx	DAO committee address. Used for Treasury, Security, or Ecosystem committees.

Multisig Wallets		
sym	synm1xxx	General multisig wallet for shared fund control. Enforces threshold signatures (e.g. 3-of-5, 5-of-7).
synu	synu1xxx	DAO treasury multisig account. Used by DAO for treasury operations or community grants.
synl	synl1xxx	Validator or council multisig wallet. For emergency rollback or governance fund management.
Fee Collector		
syntf	syntf1xxx	Central address for collecting network and uDEX fees. Automatically distributes revenue to DAO treasury and staking pools.
Network Burn Address		
syne	synergy000000000000000000000000burn	Canonical unspendable burn address recognized network-wide. Ensures permanent token destruction under the deflationary model.
Reserved Prefixes (Not in Active Use)		
synr	synr1xxx	Reserved. Possible future use for encrypted message routing or ephemeral key exchange.
syni	syni1xxx	Reserved. Potential internal identity registry or validator synchronization IDs.
synp	synp1xxx	Reserved. Could be used for privacy-enhanced transactions or permissioned subnets.

1.3 Non-Prefix Identifiers

Not all Synergy Network identifiers use a Bech32m prefix. The following special-purpose identifier types use distinct formats:

Identifier Type	Format	Description
Synergy Naming System (SNS)	<code>alice.syn</code>	Human-readable alias mapped to a wallet or UMA address. Registered on-chain via the SNS smart contract; resolves to a Synergy address.
Universal Meta-Address (UMA)	<code>syn:sYnQ3d5...:sol:EmTJ4...</code>	Multi-chain identity mapping linking a Synergy address and external chain addresses. Secured by threshold cryptography (FROST/TSS); supports key rotation and recovery.
Public Key	<code>6fd47f3a8dca7...f13fd0a937a2</code>	Hexadecimal representation of the FN-DSA-1024 public key. Used for signature verification and transaction validation. See Part IV for full hex encoding rules.

1.4 Encoding and Key Generation

All prefixed addresses follow Bech32m encoding as defined in BIP-350. The human-readable part (HRP) is the prefix listed in Section 1.2. The data portion encodes the public key hash or identifier payload. All addresses are derived from FN-DSA-1024 quantum-safe keypairs.

Authorization: Unauthorized modification or reuse of address prefixes is prohibited without DAO approval. All changes to the prefix registry must be accompanied by a formal SNTS amendment.

Part II — Denomination & Precision (SNTS-04)

2.1 Purpose and Scope

This Part defines the official denomination hierarchy, precision model, and cross-chain decimal conversion rules for the Synergy Token (SNRG), the native asset of the Synergy Network. These rules govern the Synergy Network L1, AIVM smart contracts, all wallets, explorers, SDKs, all SXCP multi-chain integrations, and off-chain systems such as exchanges and price oracles.

This standard adopts the Wei-class naming system and establishes nano-wei (nWei) as the smallest indivisible monetary unit.

2.2 Monetary Parameters

Parameter	Value
Native Asset	SNRG (Synergy Token)
Base Denomination	SNRG
Precision	9 decimal places
Smallest Unit	nWei (nano-wei)
Maximum Total Supply	12,000,000,000 SNRG (12 billion SNRG)
Maximum Total Supply (nWei)	12,000,000,000,000,000,000 nWei (12×10^{18} nWei)

All protocol-level amounts must be stored as integer nWei values. Floating-point arithmetic is strictly prohibited in consensus systems.

2.3 Denomination Hierarchy

Synergy Network uses a clean, deterministic Wei-class hierarchy. "Wei" is pronounced "way" (named after Wei Dai): mWei = milli-way, μ Wei = micro-way, nWei = nano-way.

Unit Name	Symbol	Factor	nWei Equivalent	Formal Decimal
Synergy Token	SNRG	1	1,000,000,000	10^9
milli-wei	mWei	0.001	1,000,000	10^6
micro-wei	μ Wei	0.000001	1,000	10^3
nano-wei	nWei	0.000000001	1	10^0

2.4 Accounting Rules

2.4.1 Ledger Storage

All balances, transfers, state values, and AIVM contract fields must store monetary amounts as raw nWei integers.

```
// Example
35.125 SNRG →  $35.125 \times 10^9 = 35,125,000,000$  nWei
```


2.4.2 Smart Contract Arithmetic (AIVM)

- Integer nWei arithmetic only.
- Deterministic division with explicit rounding (floor on precision reduction).
- No floating-point operations permitted.
- All user-facing values must be converted from integer nWei at the display layer.

2.4.3 Wallet & Explorer Display

- Default display unit: SNRG.
- Advanced mode units: mWei, μ Wei, nWei.
- Gas fees: always shown in nWei with an optional SNRG equivalent.

2.5 Gas Model

Gas price is denominated in nWei. Gas limit is a plain integer. The fee formula is:

```
fee_nWei = gasLimit * gasPrice_nWei
fee_SNRG = fee_nWei ÷ 109

// Example: gasLimit = 25,000 | gasPrice = 40 nWei
fee = 25,000 × 40 = 1,000,000 nWei = 0.001 SNRG
```

2.6 Cross-Chain Precision Mapping

Synergy Network is bridgeless. SXCP does not wrap, mint, or burn assets. Decimal conversion rules below apply to UX, wallets, explorers, multi-chain pricing displays, CEX integrations, oracles, and off-chain accounting systems only.

Let D_s = decimals of source chain, D_t = decimals of target chain. The general conversion formula is:

```
amount_target = amount_source × 10(D_target - D_source)

// If the exponent is negative, use floor division for determinism.
```

Conversion	Source Decimals	Target Decimals	Formula
SNRG → EVM chains	9	18	$\text{evm_amount} = \text{snrg_amount} \times 10^9$
EVM chains → SNRG	18	9	$\text{snrg_amount} = \text{evm_amount} \div 10^9$ (floor)
SNRG → Solana (9-decimal)	9	9	$\text{target_amount} = \text{source_amount}$ (no scaling)
SNRG → Bitcoin family	9	8	$\text{btc_amount} = \text{snrg_amount} \div 10$ (floor)
Bitcoin family → SNRG	8	9	$\text{snrg_amount} = \text{btc_amount} \times 10$

2.7 SDK Constants

2.7.1 Rust

```
pub const SNRG_DECIMALS: u32 = 9;

pub const NWEI_PER_SNRG: u128 = 1_000_000_000;
pub const MWEI_PER_SNRG: u128 = 1_000_000;
pub const UWEI_PER_SNRG: u128 = 1_000;

pub const EVM_DECIMALS: u32 = 18;
pub const BTC_DECIMALS: u32 = 8;
```

2.7.2 JavaScript

```
const SNRG_DECIMALS = 9n;
const NWEI_PER_SNRG = 1_000_000_000n;

const toEvm      = amount => amount * 10n ** 9n;
const fromEvm    = amount => amount / 10n ** 9n;

const toBitcoin  = amount => amount / 10n;
const fromBitcoin = amount => amount * 10n;
```

2.8 Compliance Requirements

To be Synergy-compliant, all implementations must satisfy the following:

- Store monetary values as integer nWei.
- Use nWei for all gas and fee calculations.
- Apply deterministic flooring when reducing precision.
- Display SNRG by default in wallets and explorers.
- Support mWei, μ Wei, and nWei in advanced UI modes.
- Use the conversion rules in Section 2.6 for all external decimal mappings.
- Use no floating-point arithmetic inside AIVM or node logic.

Part III — Post-Quantum Cryptography Naming Standards (SNTS-06)

3.1 Overview and Scope

This Part defines the canonical naming and formatting conventions for all NIST-standardized post-quantum cryptographic algorithms used within the Synergy Network and its associated software. These conventions ensure that algorithm names are written consistently across documentation, code comments, source files, and filenames — eliminating ambiguity between formal NIST names and legacy algorithm names.

The Synergy Network implements a six-family post-quantum algorithm suite through the Aegis PQC Core subsystem. Two of the six families — SLH-DSA and CMCE-KEM — are included in the Aegis specification as reserve algorithms but are not used in active protocol operations due to operational constraints (signature size and key size respectively). The naming conventions in this Part apply to all six families regardless of their active/reserve status.

3.2 General Rule: Documentation Form vs. Code Form

Every algorithm has two canonical forms:

- Documentation form: used in all written documentation, inline code comments, API reference text, and user-facing strings. This is the official NIST FIPS name with standard hyphenation and capitalisation.
- Code form: used in all source file names, variable names, function names, struct/type names, and other code identifiers. This is a lowercase, hyphen-free version derived directly from the documentation form.

Legacy algorithm names (CRYSTALS-Kyber, CRYSTALS-Dilithium, Falcon, SPHINCS+, HQC, Classic-McEliece) must not be used in any new documentation or code. Existing references to legacy names should be updated on next revision.

3.3 ML-KEM (Key Encapsulation — formerly CRYSTALS-Kyber)

ML-KEM is the primary key encapsulation mechanism in the Synergy Network. It is used for secure channel establishment between validators, session key derivation, and entropy seed generation.

Documentation / Comment Form	Code / Filename Form	Security Level	Recommended Use Case
ML-KEM-512	mlkem512	Level 1	Optimized for resource-constrained devices and IoT environments requiring efficient key exchange with strong baseline post-quantum protection.
ML-KEM-768	mlkem768	Level 3	Standard enterprise-grade configuration for general-purpose encryption and secure channel establishment. Default choice for Synergy validator communications.
ML-KEM-1024	mlkem1024	Level 5	Highest-security configuration for critical infrastructure, national-security, or long-term data confidentiality applications. Used in the Aegis entropy beacon.

3.4 ML-DSA (Digital Signatures — formerly CRYSTALS-Dilithium)

ML-DSA is the primary digital signature algorithm in the Synergy Network. It is used for transaction authentication, validator attestations, block proposals, governance votes, and cross-chain proofs. ML-DSA-65 is the default operational variant.

Documentation / Comment Form	Code / Filename Form	Security Level	Recommended Use Case
ML-DSA-44	mldsa44	Level 2	Suitable for embedded systems, firmware signing, or applications requiring compact signatures with moderate security.
ML-DSA-65	mldsa65	Level 3	Balanced performance for most production environments, including blockchain smart-contract signing and digital identity systems. Primary operational variant for Synergy Network.
ML-DSA-87	mldsa87	Level 5	Maximum-security configuration ideal for root certificate authorities, secure boot chains, and long-term signature validation.

3.5 FN-DSA (Digital Signatures — formerly Falcon)

FN-DSA provides high-speed, compact signatures. Within Synergy Network, FN-DSA is employed for latency-sensitive operations where ML-DSA throughput is a bottleneck. FN-DSA-1024 is also the key generation standard for all Synergy address types (see Part I).

Documentation / Comment Form	Code / Filename Form	Security Level	Recommended Use Case
FN-DSA-512	fndsa512	Level 1	High-speed, compact signature scheme for performance-critical applications and TLS certificate signing.
FN-DSA-1024	fndsa1024	Level 5	Enhanced-security configuration for blockchain validators, firmware integrity verification, and secure software distribution. Standard for Synergy address key generation.

3.6 SLH-DSA (Hash-Based Signatures — formerly SPHINCS+)

SLH-DSA provides signatures whose security rests entirely on hash function properties with no algebraic structure assumptions. It is included in the Aegis specification as a designated reserve signature scheme. SLH-DSA is not used in active Synergy Network protocol operations due to its signature size (8 KB–50 KB), which is incompatible with high-throughput consensus. It is available as an emergency fallback in the event that both ML-DSA and FN-DSA are simultaneously compromised.

SLH-DSA activation in Synergy Network requires explicit DAO governance authorization and protocol-level parameter adjustments.

Documentation Form	Code Form	Security Level	Recommended Use Case
SLH-DSA-SHAKE-128f	slhdsa-shake128f	Level 1	Fast-path variant for constrained systems or low-latency signature verification.
SLH-DSA-SHAKE-128s	slhdsa-shake128s	Level 1	Compact-signature variant optimized for bandwidth-sensitive applications.

Documentation Form	Code Form	Security Level	Recommended Use Case
SLH-DSA-SHAKE-192f	slhdsa-shake192f	Level 3	Balanced trade-off between speed and size for enterprise authentication or firmware signing.
SLH-DSA-SHAKE-192s	slhdsa-shake192s	Level 3	Reduced-size variant for embedded or mobile environments with moderate throughput.
SLH-DSA-SHAKE-256f	slhdsa-shake256f	Level 5	High-assurance signature variant for critical security systems requiring maximum robustness.
SLH-DSA-SHAKE-256s	slhdsa-shake256s	Level 5	Space-efficient configuration providing top-tier post-quantum resilience for archival or root-of-trust usage.
SLH-DSA-SHA2-128f	slhdsa-sha2128f	Level 1	SHA-2 hash family variant — fast-path, constrained systems.
SLH-DSA-SHA2-128s	slhdsa-sha2128s	Level 1	SHA-2 hash family variant — compact-signature, bandwidth-sensitive.
SLH-DSA-SHA2-192f	slhdsa-sha2192f	Level 3	SHA-2 hash family variant — enterprise authentication or firmware signing.
SLH-DSA-SHA2-192s	slhdsa-sha2192s	Level 3	SHA-2 hash family variant — embedded or mobile with moderate throughput.
SLH-DSA-SHA2-256f	slhdsa-sha2256f	Level 5	SHA-2 hash family variant — critical security systems, maximum robustness.
SLH-DSA-SHA2-256s	slhdsa-sha2256s	Level 5	SHA-2 hash family variant — archival or root-of-trust.

3.7 HQC-KEM (Code-Based KEM — formerly Hamming Quasi-Cyclic)

HQC-KEM is a code-based key encapsulation mechanism whose security rests on a hardness assumption independent of lattice problems. Within Aegis, HQC-KEM serves as an active reserve KEM alongside ML-KEM, providing algorithm diversity.

Documentation / Comment Form	Code / Filename Form	Security Level	Recommended Use Case
HQC-KEM-128	hqckem128	Level 1	Lightweight configuration for constrained devices and fast ephemeral key exchange.
HQC-KEM-192	hqckem192	Level 3	General-purpose variant providing robust post-quantum protection for enterprise systems and distributed ledgers.
HQC-KEM-256	hqckem256	Level 5	Maximum-security configuration for classified or long-term high-sensitivity data.

3.8 CMCE-KEM (Classic-McEliece — Code-Based KEM)

CMCE-KEM's security rests on binary Goppa code decoding — a hardness assumption entirely distinct from both module lattice problems and quasi-cyclic code problems. CMCE-KEM is included in the Aegis specification as the primary reserve KEM but is not used in active Synergy Network operations due to its large public key size (261 KB–1,044 KB), which is incompatible with protocol-frequency key exchange. It provides an emergency encapsulation pathway in the event that both ML-KEM and HQC-KEM are simultaneously compromised.

CMCE-KEM activation requires DAO governance authorization.

Documentation / Comment Form	Code / Filename Form	Security Level	Recommended Use Case
CMCE-KEM-348864	cmce348864	Level 1	Lightweight applications requiring baseline post-quantum security with smaller key sizes.
CMCE-KEM-460896	cmce460896	Level 3	Balanced performance and higher security margin for general-purpose KEM in production systems.
CMCE-KEM-6688128	cmce6688128	Level 5	Maximum-security configuration for high-assurance or long-term archival protection.
CMCE-KEM-6960119	cmce6960119	Level 5	Alternative high-security variant providing cryptanalytic diversity within Level 5.
CMCE-KEM-8192128	cmce8192128	Level 5	Ultra-high-security configuration offering strongest resistance against quantum and classical attacks; for root-of-trust or foundational cryptographic anchors.

Part IV — Binary & Hexadecimal Encoding Conventions (SNTS-07)

4.1 Scope

This Part governs how raw binary data — cryptographic hashes, public keys, signatures, and other byte arrays — is serialized and displayed across all Synergy Network contexts: JSON APIs, documentation, logs, on-chain storage, and inter-process communication. Address encoding is governed separately by Part I (Bech32m); this Part covers all other binary objects.

4.2 General Hex Encoding Rules

All binary data, when represented as text, uses lowercase hexadecimal encoding with no prefix. The following rules are normative:

- Characters must be from the set [0-9a-f] only — uppercase hex characters [A-F] are not permitted in any Synergy-native encoding context.
- No prefix of any kind is prepended to hex strings. The 0x prefix convention is not used anywhere in the Synergy protocol.
- Hex strings are always full-length and zero-padded to their expected byte-count $\times 2$ characters. Leading bytes of zero value are never omitted.
- The character length of a serialized hex string is always exactly: $\text{byte_length} \times 2$.

The 0x prefix, mixed-case checksummed hex (EIP-55 style), Base58, Base58Check, and Base64 are not used in any Synergy-native encoding context. These formats may appear only at external-chain compatibility boundaries and must be explicitly marked as compatibility translations.

4.3 Typed Hash Identifiers

Protocol-level hashes that appear in user-facing contexts (explorers, APIs, log output) carry a typed prefix that identifies the entity type the hash represents. The prefix convention established by the protocol is `syn{type}-{hex}`. Prefixed and raw hex forms each have their appropriate context:

- Prefixed form (e.g. `syntxn-a0d53ef9...`): used in user-facing contexts — explorer UIs, API responses, documentation examples, and log messages where the entity type must be unambiguous.
- Raw hex form (e.g. `a0d53ef9...`): used in machine-to-machine contexts such as internal node state, binary serialization, and inter-process communication where the type is unambiguous from structure.

Identifier	Prefix	Hex Length	Represents
Transaction hash	<code>syntxn-</code>	64 chars	Hash digest of the serialized transaction.
Block hash	<code>synblk-</code>	64 chars	Hash digest of the block header.
Merkle root	<code>synmkl-</code>	64 chars	Merkle root of the transactions contained in a block.
Epoch randomness beacon	<code>synepoch-</code>	128 chars	SHA3-512 entropy beacon output at an epoch boundary.
Quorum certificate hash	<code>synqc-</code>	64 chars	Hash of a finalized Quorum Certificate.

64 hex characters = 32 bytes, consistent with SHA3-256 output. 128 hex characters = 64 bytes, consistent with SHA3-512 output used in the PoSy entropy beacon.

4.4 Public Key Serialization

Public keys are serialized as lowercase hex when represented as text. No prefix or wrapper is applied. The expected character count is determined by the algorithm variant and is fixed — implementations must validate length on deserialization. The table below lists expected sizes for the active Synergy protocol variants:

Key Type	Bytes	Hex Characters	Context
ML-KEM-768 public key	1,184	2,368	Validator session key exchange
ML-KEM-1024 public key	1,584	3,168	Entropy beacon / high-security channels
ML-DSA-65 verification key	1,952	3,904	Default transaction and attestation signing
ML-DSA-87 verification key	2,432	4,864	Root-of-trust and governance signing
FN-DSA-1024 verification key	≈ 1,793	≈ 3,586	Address key generation (see Part I)

4.5 Signature Serialization

Signatures are not displayed in user-facing interfaces. In API responses and log output where a signature must be serialized to text, lowercase hex without prefix is used. Signatures are never Base64-encoded in Synergy-native contexts. The expected byte sizes for reference are: ML-DSA-65 ≈ 3,309 bytes; ML-DSA-87 ≈ 4,627 bytes; FN-DSA-512 ≈ 666 bytes; FN-DSA-1024 ≈ 1,280 bytes.

4.6 Encoding Formats Explicitly Not Used

To prevent ambiguity, the following encoding formats are explicitly prohibited in all Synergy-native contexts:

Format	Rule
0x-prefixed hex	Not used. The 0x prefix is an Ethereum convention and is not part of the Synergy encoding system.
Uppercase or mixed-case hex	Not used. All hex is strictly lowercase [0-9a-f].
Base64	Not used in Synergy-native contexts. May appear only in external compatibility layers.
Base58 / Base58Check	Not used. These are Bitcoin-family encoding conventions.
EIP-55 checksummed addresses	Not used. Mixed-case checksumming is an Ethereum-specific convention incompatible with the Synergy address system.

Part V — Timestamp Representation (SNTS-08)

5.1 Wire Format

All timestamps in the Synergy protocol are Unix epoch timestamps expressed as an unsigned 64-bit integer (u64) counting whole seconds since 1970-01-01T00:00:00Z. Millisecond, microsecond, and nanosecond precision is not used at the protocol layer. Fractional seconds are never transmitted.

```
// Rust wire type
timestamp: u64    // whole seconds since Unix epoch

// JSON API type
"timestamp": 1741561800    // integer, never a float or string
```

5.2 Precision Rationale

Second-level precision is sufficient for all Synergy protocol operations. The minimum meaningful time unit in the PoSy consensus protocol is the block interval (approximately 2 seconds). The replay-attack prevention window for inter-cluster messages is 5 seconds. Sub-second precision would add wire overhead with no protocol benefit and would introduce unnecessary dependency on clock synchronization quality across heterogeneous validator hardware.

Millisecond timestamps (the JavaScript `Date.now()` convention and the Ethereum block timestamp convention in some tooling) are not used. Implementations that receive a millisecond-scale integer should reject it as malformed rather than silently accepting a timestamp 1,000× larger than expected.

5.3 Display Format in Documentation and UI

When timestamps are shown to humans — in documentation examples, block explorer displays, or log output — the canonical display format is ISO 8601 UTC with a Z suffix and no fractional seconds:

```
2026-03-10T14:30:00Z

// NOT:  2026-03-10T14:30:00.000Z    (no fractional seconds)
// NOT:  2026-03-10T09:30:00-05:00   (no local timezone offsets)
// NOT:  March 10, 2026              (no informal date strings)
```

Local time representations must not be used in any protocol-facing display. All times are UTC, always.

5.4 Timestamp Field Naming Conventions

Timestamp fields in all protocol data structures and API responses follow a consistent naming scheme:

Context	Convention	Examples
Current/single timestamp	timestamp	timestamp, block_timestamp
Event time	{event}_at	created_at, finalized_at, registered_at
JSON field names	snake_case	"created_at": 1741561800
Rust field names	snake_case	pub created_at: u64

Fields must never use millisecond-suggesting suffixes such as `timestampMs`, `ts_ms`, or `timestamp_millis`. If a field name contains "ms" it implies the value is in milliseconds — which is not a valid Synergy timestamp format.

Part VI — Network Environment Identifier Conventions (SNTS-09)

6.1 Canonical Environment Names

The Synergy Network operates across five named environments in the following progression from development to production. Each environment has a canonical technical identifier that is used in all code, configuration, and system contexts, and a display name for prose and headings.

Display Name	Technical Identifier	Chain ID
Devnet	devnet	338638
Testnet Beta	testbeta	338639
Testnet	testnet	338640
Mainnet Beta	mainbeta	338641
Mainnet	mainnet	338642

6.2 Usage Rules by Context

The technical identifier is the single canonical form used in all system contexts. The display name appears only in prose body text and section headings, where title case is appropriate.

Context	Rule	Examples
Code identifiers, config keys, env vars	Lowercase technical identifier	SYNERGY_NETWORK=testbeta
DNS subdomains	{identifier}-{service}.synergy-network.io	testbeta-core-rpc.synergy-network.io
Nginx upstream names	{identifier}_{service}	testbeta_core_rpc
Prose / documentation body	Lowercase technical identifier	"the testbeta environment"
Section headings only	Display name, title case	"Testnet Beta Configuration"
Abbreviations	Never permitted	tb, mb, dev, tn, mn are not valid identifiers

6.3 Port Conventions

Network service ports follow a structured scheme based on environment. The base port for each service class is incremented by the environment offset:

Service	devnet	testbeta	testnet	mainbeta	mainnet
L1 Node RPC	38638	38648	38658	38668	38678
Core RPC	48638	48648	48658	48668	48678
Core WS	58638	58648	58658	58668	58678

Port assignments for testbeta, testnet, mainbeta, and mainnet are to be confirmed against the network configuration specification when those environments are provisioned.

6.4 Global Configuration

The identifier global designates configuration that applies across all environments: shared DNS roots, TLS policies, and shared nginx rules. It is not an operational environment — nodes do not join or sync against "global" — but it appears as a named configuration section alongside the five environment identifiers.

Appendix A — Quick Reference

This appendix provides a condensed reference for the most commonly needed naming and encoding conventions across all Parts of this document.

A.1 Address Prefixes (Selected)

Prefix	Type	Use
synw	Wallet	Standard user wallet
syntxn	Transaction	On-chain PoSy transaction
synxxn	Cross-chain Tx	SXCP cross-chain transaction
synv1-synv5	Node	Node address (Class I–V)
syndao	Governance	DAO proposal
synq / sync	Contract	System / custom smart contract

A.2 Denomination Units

Unit	Symbol	nWei Value
Synergy Token	SNRG	1,000,000,000
milli-wei	mWei	1,000,000
micro-wei	µWei	1,000
nano-wei (base)	nWei	1

A.3 PQC Algorithm Identifiers (Active Protocol)

Algorithm	Active Variants	Default for Synergy
ML-KEM	ML-KEM-768, ML-KEM-1024	Validator key exchange / entropy beacon
ML-DSA	ML-DSA-65, ML-DSA-87	Transaction signing / attestations (default: ML-DSA-65)
FN-DSA	FN-DSA-1024	Address key generation; latency-sensitive signing
HQC-KEM	HQC-KEM-192	Reserve KEM (algorithm diversity)

A.4 Hex Encoding Rules at a Glance

- Always lowercase [0-9a-f].
- Never use 0x prefix.
- Always full-length, zero-padded.
- Typed prefixes for user-facing hashes: syntxn-, synblk-, synmkl-, synepoch-, synqc-.

A.5 Timestamp Rules at a Glance

- Wire format: u64 integer, whole seconds since Unix epoch.
- Display format: 2026-03-10T14:30:00Z (ISO 8601 UTC, no fractional seconds).
- Field naming: timestamp / {event}_at in snake_case.

A.6 Network Environments at a Glance

Display Name	Identifier	Chain ID
Devnet	devnet	338638
Testnet Beta	testbeta	338639
Testnet	testnet	338640
Mainnet Beta	mainbeta	338641
Mainnet	mainnet	338642